EXPERIMENT - 1

LINUX COMMANDS

6. AIM:
To study various network commands used in Linux and windows.

1. CODE:
```
arp -a
```

OUTPUT:
```
(172.16.11.109) at 00:27:0e:13:f7:8c [ether] on enp3s0
(172.16.8.149) at 00:27:0e:13:f3:fe [ether] on enp3s0
(172.16.8.186) at 7c:57:58:38:e6:24 [ether] on enp3s0
(172.16.9.212) at 7c:57:58:38:da:a3 [ether] on enp3s0
```

2. CODE:
```
hostname
```

OUTPUT:
```
localhost.localdomain
```

3. CODE:
```
netstat -r
```

OUTPUT:
Kernel IP routing table

Destination	Gateway	Genmask	Flags	MSS/Window	irtt	Iface
default	gateway	0.0.0.0	UG	0	0	0 enp3s0
172.16.8.0	0.0.0.0	255.255.252.0	U	0	0	0 enp3s0

4. CODE:
nslookup www.facebook.com

OUTPUT:

Server: 172.16.8.1

Address: 172.16.8.1#53

Non-authoritative answer:

www.facebook.com

Canonical name = star-mini.c10r.facebook.com

Name: star-mini.c10r.facebook.com

Address: 57.144.56.1

5. CODE:
route

OUTPUT:

Kernel IP routing table

Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface
default	gateway	0.0.0.0	UG	100	0	0	enp3s0
172.16.8.0	0.0.0.0	255.255.252.0	U	100	0	0	enp3s0

6. CODE:
ip address show

OUTPUT:

1: lo: inet 127.0.0.1/8 scope host lo

2: enp3s0: inet 172.16.8.127/22 brd 172.16.11.255
scope global enp3s0

7. ifconfig

OUTPUT:

emp3s0 : flags = 4163 <UP, BROADCAST, RUNNING MULTICAST>
mtu 1500 0

inet 172.16.11.207

netmare 255.255.252.0

broadcast. 17.16.8.255

8. mtr - serves as a network diagonistic tool

OUTPUT:

mtr google.com

local host : local domain ()

Host

Packets

loss %	snt	lost	Avg	Boot	wots	Stdev
0.01 %	17.8	0.1	0.1	0.0	0%	0.0

9. tcpdump - designed to capture and display packets

output:

tcpdump - D

1. enp3s0 (up, Running)
2. any [pseudo: device that captures on all interfaces) [up, running]
3. lo (up, Running, Loopback)
4. wlp 3T0
5. Bluetooth 0 (Bluetooth adapter number 0)

10. nbtstat -r

OUTPUT:

NetBIOS Names Resolution and Registration Statistics

Resolved by Broadcast = 0

Resolved by Name Server = 0

Registered by Broadcast = 27

Registered by Name server = 0

11. pathping -4 172.16.97.153

OUTPUT:

over a max. of 30 hops:

0 Sg007 [172.16.97.153]
1 Sg007 [172.16.97.153]

Computing statistics for 25 seconds

Hop	RTT	Source to Here This Node/Link Lost/sent = Pct	Address
0		0/100 = 0%	172.16.97.153
1	0ms	0/100 = 0%	172.16.97.153

Trace Complete

RESULT:

Thus the network commands using Linux and windows are successfully executed and the output is verified.

EXPERIMENT-2

AIM: To study about different types of network cables.

Different types of cables used in networking are:

1. Unshielded Twisted Pair (UTP) Cable
2. Shielded Twisted Pair (STP) cable
3. Coaxial Cable
4. Fibre Optic Cable.

Cable Type: UTP



Category	Max. Data Transmission
Category 3	10 bps
Category 5	Up to 100 Mbps
Category 5e	1 Gbps

Advantages: Cheaper in cost

Easy to install as they have a small overall diameter.

Disadvantages: More prone to Electromagnetic Interference and noise.

Application / Use: 10-Base-T Ethernet
Fast Ethernet, Gigabit Ethernet.

Cable type: STP

Category	Max. Data Transmission
Category 6, 6a	10 Gbps
Category 7	10 Gbps.



Advantages: Shielded
Faster than UTP
less susceptible to noise and interference.

Disadvantages: Expensive
Greater Installation effort.

Application / Use:

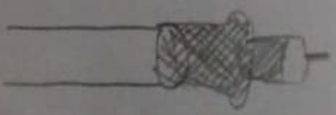
Category 6/6a ⇒ Gigabit ethernet, 10G Ethernet (55 m) widely used in data centres.

Category 7 ⇒ Gigabit ethernet, 10G Ethernet (100m)

Cable Type: Coaxial Cable

Category: RG-6, RG-59, RG-11

Max. Data Transmission: 10-100 Mbps



Advantages:

- High bandwidth
- Immune to interference
- Low loss bandwidth
- Versatile

Disadvantages:
- Limited distance
- Cost
- Size is bulky

Cable Type: Fibre Optics Cable

Category:
- Single Mode
- Multimode

Max. Data Transmission: 100 Gbps

Advantages:
- High speed & bandwidth
- High Security

Disadvantages:
- Expensive
- Requires skilled Installers.

Application / use: Maximum distance of fibre optics cable is around 100 metres.

RESULT:

Thus the study about different type of network cables are executed successfully.

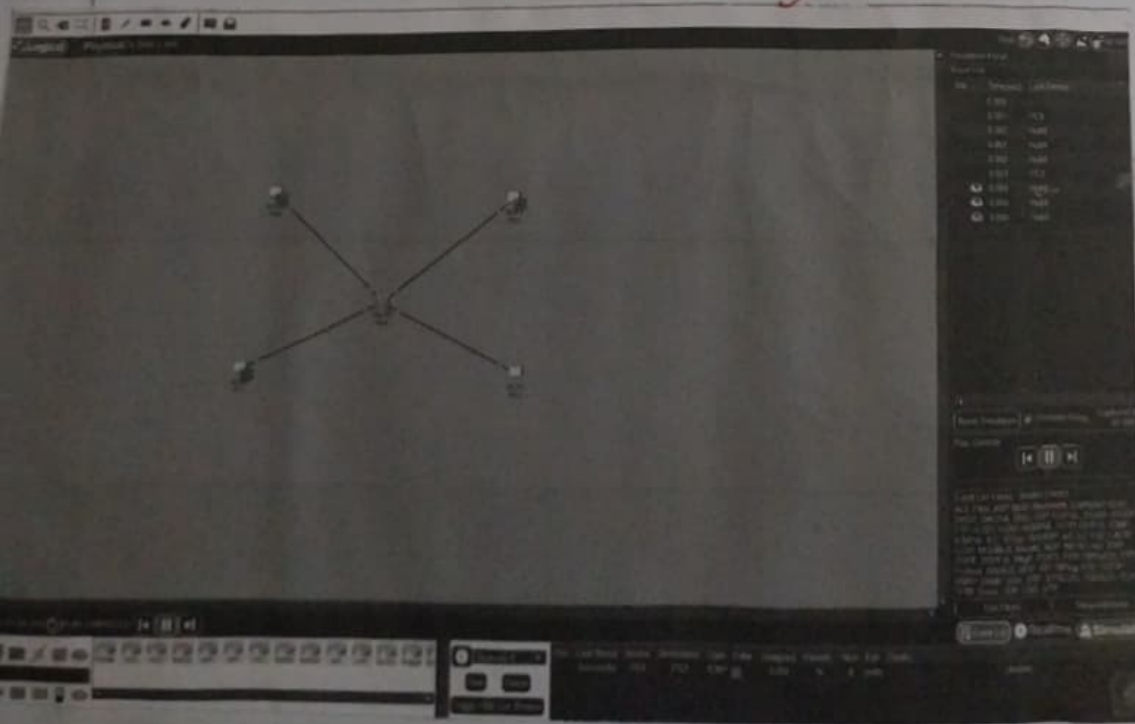
EXPERIMENT-3

AIM: To study the packet tracer tool installation and user interface overview.

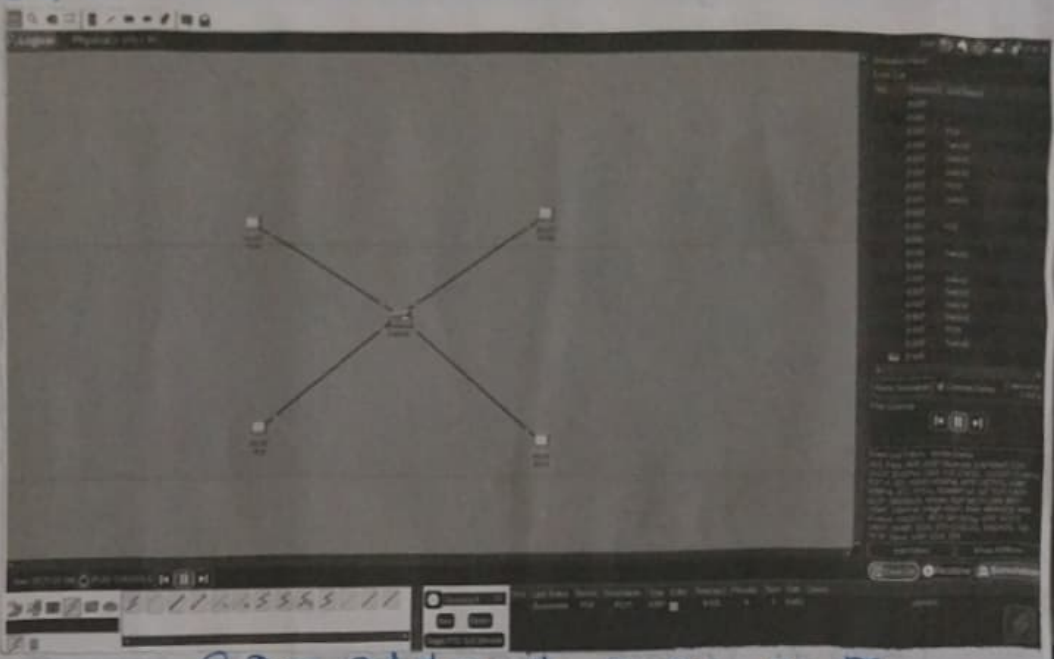
Analyse the behaviour of network devices using Cisco Packet Tracer Simulator.

1. From the network component box, click and drag-and-drop the below components:
   a. Generic PCs and One HUB
   b. Generic PCs and One Switch

2. Click on Connections:
   a. Click on copper straight through cable
   b. Select one of the PC and connect it to HUB using the cable. The link LED glow green indicating that the link is up, similarly connect remaining 3 PC's to the hub.
   c. Similarly connect 4 PCs to the switch using copper-straight-through cable.



3. Click on the PCs connected to hub, go to the Desktop tab, click on IP Configuration, and enter an IP address and subnet mask. Here, the default gateway and DNS server information is not needed as there are only end devices in the network.



4. Drag and drop it on one of PC (source machine) and then drop it on another PC (destination machine) connected to the HUB.

4. Observe the flow of PDU from source PC to destination PC by selecting the Realtime mode of simulation.

5. Repeat step #3 to step #5 for the PCs connected to the switch.

6. Observe how HUB and switch are forwarding the PDU and write your observation and conclusion about the behaviors of switch and HUB.

RESULT:

Thus, the study of packet tracer tool installation and user interface overview has been executed successfully.

EXPERIMENT - 4

AIM:

To setup and configure a LAN (Local Area network) using a Switch and Ethernet Cables in your lab.

How to set up a LAN:

Step 1: Plan and Design and appropriate network topology taking into account network requirements and equipment location.

Step 2: You can take 4 Computers, a Switch with 8, 16 or 24 ports which is sufficient for networks of these sizes, and 4 Ethernet Cables.

Step 3: Connect your computers to network switch via an Ethernet cable, which is as simple as plugging one end of the Ethernet cable into your computer and the other end into your network switch.

Step 4: Assign, IP address to your PCs

1. Log on to the client Computer as Administrator or as Owner
2. Click Network and Internet Connection
3. Right Click local Area Connection / Ethernet -> Go to properties -> Select Internet Protocol (TCP IPV4) -> Click on Properties -> Select use the following ip address option and assign ipaddress.

IP Address Configuration Table

Device	IP address	Subnet Mask	Connected To
PC0	10.1.1.1	255.0.0.0	Switch Port Fa0/1
PC1	10.1.1.2	255.0.0.0	Switch Port Fa0/2
PC2	10.1.1.3	255.0.0.0	Switch Port Fa0/3
PC3	10.1.1.4	255.0.0.0	Switch Port Fa0/4
Switch	10.1.1.5	255.0.0.0	VLAN 1 Interface

Step 5: Configure the Switch Management IP

a) Click the Switch → CLI tab

b) Enter the following commands to assign a management IP:

```
enable
configure terminal
interface vlan 1
ip address 10.1.1.5 255.0.0.0
no shutdown
exit.
```

Step 6: Verify Connectivity Using Ping

a) Click a PC → Desktop tab → Command Prompt

b) Use the ping command to test connectivity to the other PC's eg., ping 10.1.1.2

c) Successfull "Reply from..." messages confirm connectivity.

Step 7: Demonstrate Data sharing

a) Add a server device (End devices → Server-PT) and connect it to switch.

b) Assign it an IP address via its Desktop → IP Configuration tab.

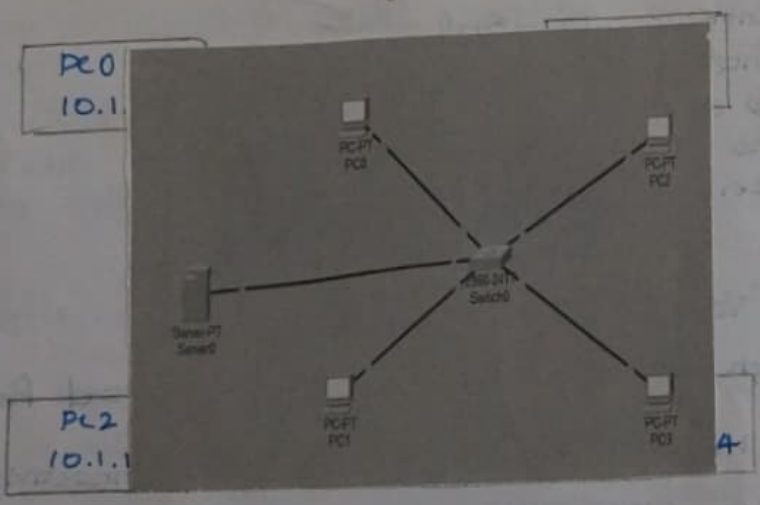
c) Open the Server's Services (Config tab) and enable the HTTP service.

d) From another PC, open the Web Browser (Desktop tab) and enter the Server's IP address to load the page, confirming successful data transfer across the LAN.

Step 8: Observe Packet Flow in Simulation Mode.

a) Switch to Simulation mode

b) Send a ping or web request between two devices and use the Play / Capture controls to step through the packet as it travels hop-by-hop through the switch, observing MAC-address-based forwarding.



RESULT:

Thus the Setting up a local Area network (LAN) has been executed successfully.

EXPERIMENT - 5

AIM:

To experiment on packet capture tool : Wireshark.

Capturing and Analysing packets using wireshark tool:

To filter, Capture, view, packets in Wireshark tool.

Capture 100 packets from the Ethernet : IEEE 802.3 LAN Interface and save it.

Procedure:

Select local Area Connection in Wireshark.
Go to capture option
Select stop capture automatically after 100 packets.
Then click start capture.
Save the packets.

1. Create a Filter to display only TCP/UDP packets, inspect the packets and provide the flow graph.

-> Select local Area Connection in Wireshark
-> Go to capture option
-> Select stop capture automatically after 100 packets.
-> Then click start capture.
-> Search TCP packets in search box.
-> To see flow graph click statistics Flow Graph.
-> Save the packets.

2. Create a filter to display only ARP packets and inspect the packets.

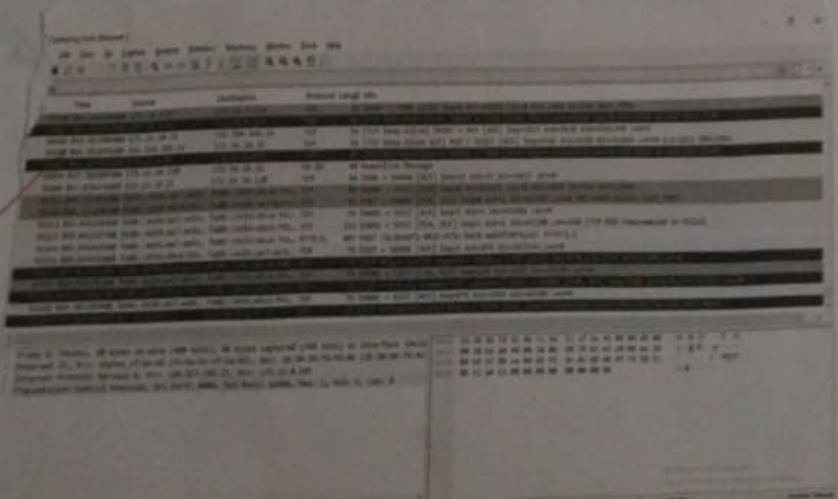
-> Go to capture option
-> Select stop capture automatically after 100 packets
->Then click start capture
->Search ARP packets in search bar.
->Save the packets.

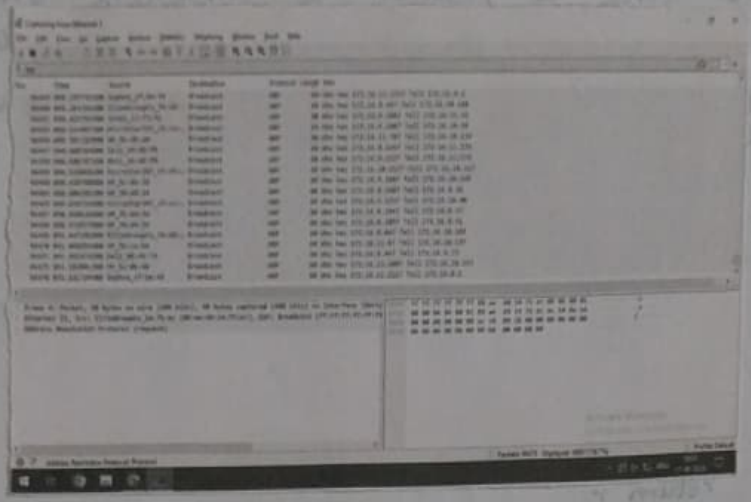
3. Create a filter to display only HTTP packets

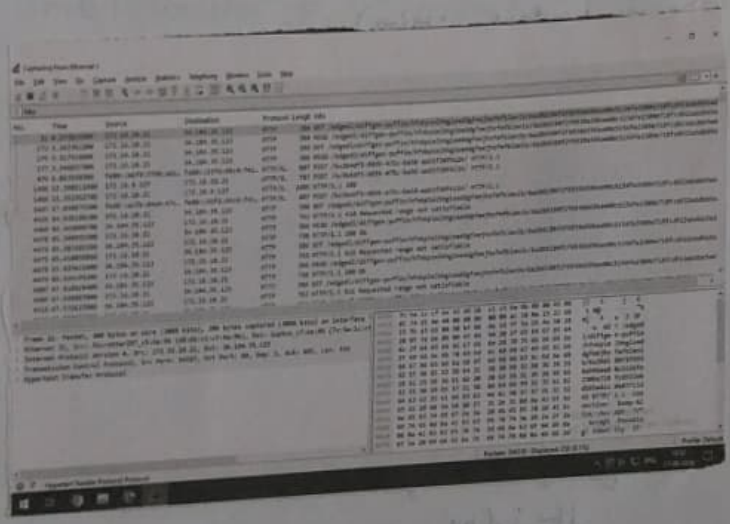
->select local Area Connection in Wireshark
-> Go to capture option
-> Select stop capture automatically after 100 packets.
-> Then click start capture.
-> Search HTTP packets in search bar.

4. Create a filter to display only IP/ICMP packets and Inspect the packets.

-> Select local Area Connection in Wireshark.
-> Go to capture option
-> Select stop capture automatically after 100 packets.
-> Then click start capture.
-> Search ICMP/IP packets in search bar.







RESULT:

Thus the experiment to access all the packets using wireshark was executed successfully.

EXPERIMENT - 6

AIM:

To write a program to implement error detection and correction using HAMMING code concept, and make a test run to input data stream and verify error correction feature.

CODE:

```
def redundant_bits(m):
    r=0
    while 2**r < m+r+1:
        r+=1
    return r

def encode(data):
    m=len(data)
    r= rendundant_bits(m)
    print("Number of data bits:", m)
    print("Number of redundant bits: ", r)

    n= m+r
    h = ['0'] *(n+1)

    j=0
    for i in range(r):
        p=2**i
        parity=0
        for j in range(1,n+1):
            if j & p:
                parity ^= int(h[j])
        h[p]= str[parity]
    print("Redundant bit positions: " [2** i for iin
                                        range(r)])
    print("Hamming Code: ", ".join(h[1:]))

    return ''. join h[1:])
```

```
def check(h):
    n= len(h)
    r=0
    while 2**r <=n :
        r+=1
    error=0
    print("\n Parity Checking: ")
    for i in range(r):
        p=2**i
        parity=0
        for j in range(1,n+1):
            if j & p:
                parity ^= int(h[j-1])
        print("Position",p,":",parity)
        if parity!=0:
            error+=p
    if error==0:
        print("\n No error detected")
    else:
        print("Error has occured")
        print("Error position:",error)

        if h[error-1]=='0':
            h[error-1]="1"
        else:
            h[error-1]='0'
        print("Error Corrected")
        print("Corrected Hamming Code:"," ".join(h))
    return h
```

```
def remove_parity(h):
    data = ''
    for i in range(1, len(h)+1):
        if i % 8 != 0:
            data += h[i-1]
    return data

def binary_to_text(data):
    text = ''
    for i in range(0, len(data), 8):
        byte = data[i:i+8]
        text += chr(int(byte, 2))
    return text

text = input("Enter data to transmit:")
binary = ''.join(format(ord(c), '08b') for c in text)

print("\nData:", text)
print("Binary:", binary)

channel = encode(binary)

with open("channel.txt", "w") as f:
    f.write(channel)

print("\nData saved in Chanel.txt")

with open("chanel.txt", "r") as f:
    received = f.read()

print("\nReceived data:")
print(received)

Change = input("\nDo you want to change the
    transmitted Data? (yes/no):")

if Change.lower() == "yes":
    pos = int(input("Enter position to Change:"))
    if pos >= 1 and pos <= len(received):
        h = list(received)
```

```
            if h[pos-1] == '0':
                h[pos-1] = '1'
            else:
                h[pos-1] = '0'

            received = ''.join(h)
            print("Changed Data :")
            print(received)
        else:
            print("Invalid Position")
            exit()

    hamming = list(received)
    hamming = check(hamming)
    data = remove_parity(hamming)
    print("\n Data after removing redundant bits :")
    print(data)
    output = binary_to_text(data)
    print("\n Output :", output)
```

INPUT: Enter data to transmit: HI
Do you want to change the transmitted data? (yes/no): no

OUTPUT: Data : HI
Binary : 0100100001001001
No. of data bits : 16
No. of redundant bits : 5
Redundant bit positions : [1, 2, 4, 8, 16]
Hamming code : 10011001100000100 1001
Data saved in channel.txt
Received Data:
10011001100000100 1001
done
Data after removing redundant bits :
0100100001001001

RESULT:

Thus the program is implemented using python for error correction & error detection using hamming code was executed successfully.

EXPERIMENT - 7

AIM:

To write a program to implement flow control at data link layer using SLIDING WINDOW PROTOCOL. Simulate the flow of frames from one node to another.

CODE:

SENDER.PY

```
import time
w=int(input("Enter Window Size:"))
msg=input("Enter Message:")
frames=[[i,ch] for i, ch in enumerate(msg)]
base=0
while base<len(frames):
    with open("Sender_Buffer.txt","w") as f:
        for i in range(base, min(base+w, len(frames))):
            f.write(f"{frames[i][0]},
                    {frames[i][1]}\n")
print("\n Frames Sent:")
for i in range(base,min(base+w,len(frames))):
    print(frames[i])
time.sleep(3)
try:
    with open("Receiver_Buffer.txt","r") as f:
        data=f.read().strip()
    if data.startswith("ACK"):
        ack=int(data.split(":")[1])
        if ack==base+w-1 or ack>=len(frames)-1:
            print("ACK received:",ack)
            base=ack+1
        else:
            print("Wrong ACK Resending...")
```

```
    elif data.startswith("NACK"):
        print("NACK Received. Resending....")

except:
    print("Waiting for Receiver.....")
```

RECEIVER.PY

```
exp = 0
with open("Sender_Buffer.txt","r") as f:
    lines = f.readlines()

ok = True
for line in lines:
    no, data = line.strip().split(",")
    if int(no) != exp:
        ok = False
        nack = exp
        break
    exp += 1

with open("Receiver_Buffer.txt", "w") as f:

    if ok:
        f.write(f"ACK: {exp-1}")
        print("ACK Sent:", exp-1)

    else:
        f.write(f"NACK: {nack}")
        print("NACK Sent:", nack)
```

INPUT:

Enter Window Size: 3

Enter Message: HELLO

Output (Sender)

Frames Sent:

[0, 'H']

[1, 'E']

[2, 'L']

Ack received: 2

Frames sent:

[3, 'L']

[4, 'O']

Ack received: 4

Output (Receiver)

Ack Sent: 2

Ack Sent: 4

RESULT:

Thus the program to implement flow control at data link layer used SLINDING WINDOW has been executed successfully.

EXPERIMENT - 8(a)

AIM:

To discover live host using NMAP scans (ARP, ICMP, TCP/UDP) on tryhackme platform.

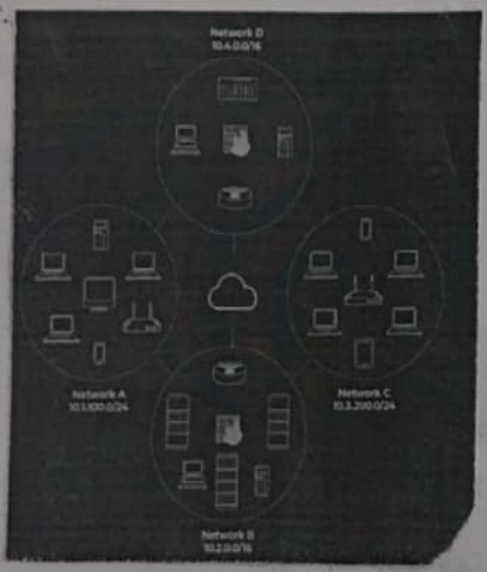
Introduction:

When targetting a network we need an efficient tool to handle repetitive tasks.

This room is first of four in NMAP series.

* NMAP live host discovery
* NMAP basic port scans
* NMAP Advanced port scans
* NMAP post port scans.

Subnetworks:



This figure displays two types of subnet.

/16 subnets

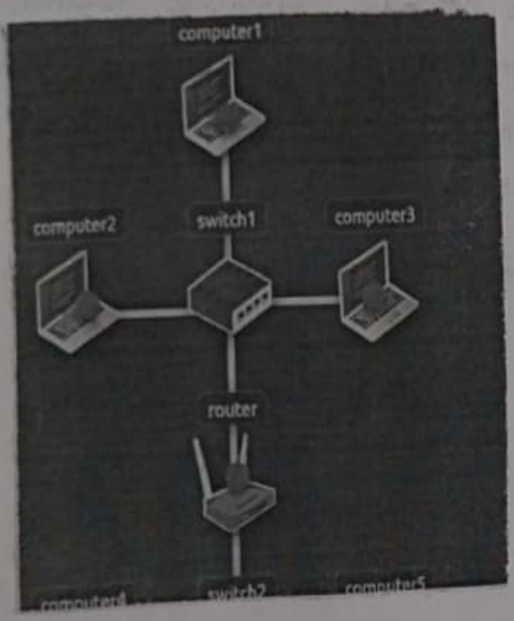
/24 subnets.

From computer 1

To computer 1

Packet type "ARP request"

Data computer 6.



Send Packet

From:
computer1

To:
computer1

Packet Type:
arp_request

Data:
computer6

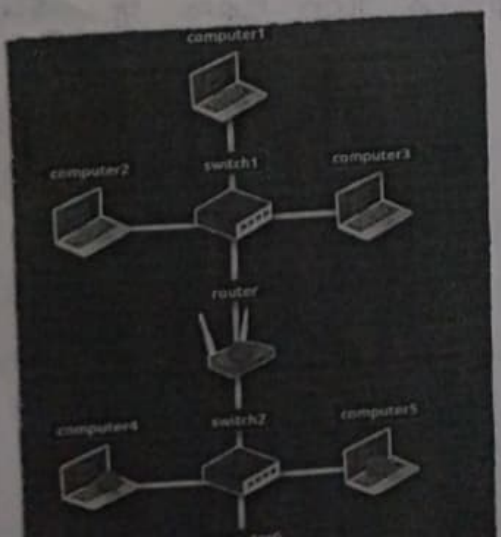
Send Packet

From computer 4

To computer 4

Packet Type: "ARP Request"

Data: computer 6.



Send Packet

From:
computer4

To:
computer4

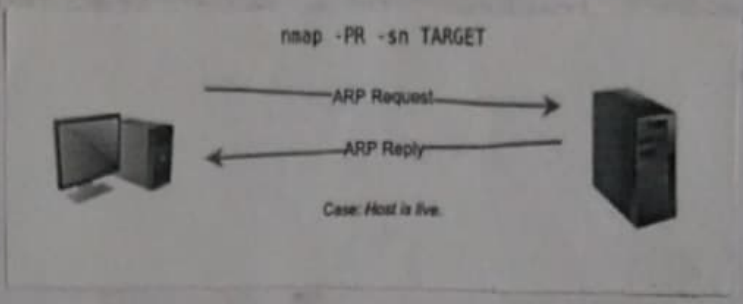
Packet Type:
arp_request

Data:
computer6

Send Packet

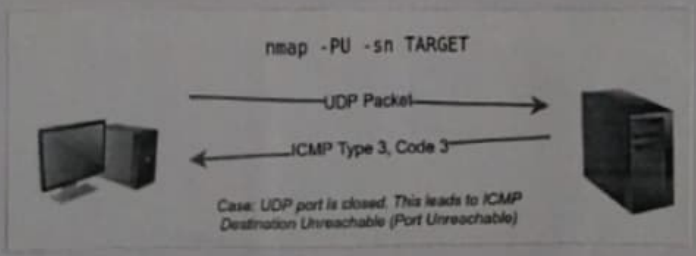
NMAP Host discovery using ARP:

When a privelaged user tries to scan targets on local network, NMAP uses ARP request.



UDP PING:

While receiving UDP packets to open UDP ports may not trigger a response, sending them to closed UDP ports.



RESULT:

Thus the requirement has been completed successfully using tryHackme.

EXPERIMENT-9

AIM:

Implementation of Subnetting in CISCO Packet tracer Simulator.

Creating a network topology:

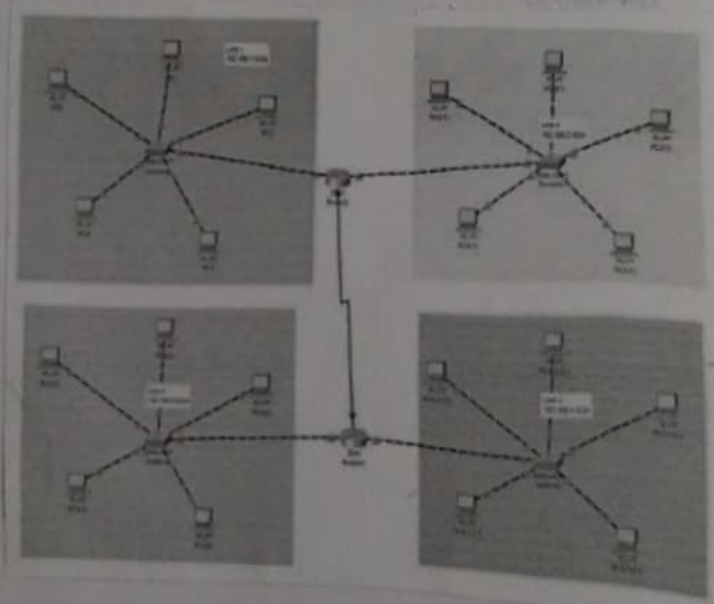
To create a network topology in packet trace, select "New" button in top, then select network & generic.

Adding the devices:

To add a device, select the device from bottom left and drag it onto network topology.

Subnetting:

To subnet the network address of 192.168.1.0/24 to provide enough space for at least 5 address for end device, switch & router.



configuring the devices:

Right click on router and select "CLI". will open by configuring the router. Right click CLI enter following commands.

```
#enable
#configure terminal
#  Interface Fast Ethernet 0/0
#ip address {IP address} {Subnet mask}
#no shutdown
#exit
```

```
Interface Fast Ethernet 0/1
ip address {IP address} {subnet mask}
no shutdown
exit
```

TESTING THE NETWORK:

ire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num
	Successful	PC4(2)	Router2	ICMP		0.000	N	12
	Successful	PC4(2)	PC2(1)(1)	ICMP		0.000	N	13
	Successful	PC0	Router0	ICMP		0.000	N	14

RESULT:

The Implementation of subnetting in CISCO Packet tracer simulator was executed successfully.

EXPERIMENT - 10

AIM:

To simulate routing Information Protocol (RIP) using CISCO Packet tracer and observe dynamic routing between different networks.

IP configuration:

Device	Interface	IP address
PC0	Fast Ethernet	10.0.0.2/8
Router 0	Fa0/1	10.0.0.1/8
Router 0	S0/0/1	192.168.1.254/30
Router 1	S0/0/0	192.168.1.250/30
Router 1	S0/0/1	192.168.1.246/30
Router 2	S0/0/0	192.168.1.245/30
Router 2	S0/0/1	192.168.1.253/30
PC 1	Fast Ethernet	20.0.0.2/8

PROCEDURE:

1) Create the topology with 3 routers and 2 PCs.

2) Connect the routers using serial cables and connect PC's to the respective routers.

3) Assign IP addresses to all PCs and router Interface.

4) Enable the router Interfaces.

5) Configure the remaining Interface according to the given IP table.

6) Enable RIP on router 0 and configure RIP on router 1 and 2.

7. From PC0, use ping 20.0.0.2 to verify connectivity.

8. Use tracert to observe the route taken by packets.

9. Disconnect the direct Router 0 - Router 2. link and observe that RIP routes the traffic through Router 1.

RESULT:

Thus RIP, was successfully simulated in CISCO Packet Tracer, and dynamic router selection and routing were observed.

EXPERIMENT - 11

AIM:

To design and configure a simple Internetwork using a router in Cisco Packet tracer.

PROCEDURE:

1. Open Cisco Packet Tracer and Create a new topology.

2. Add 2 routers, switches and 3-4 PCs for each network.

3. Connect PCs to switches to routers using ethernet cables.

4. Assign static IP address and subnet mask to all PCs.

5. Configure IP address on router interface.

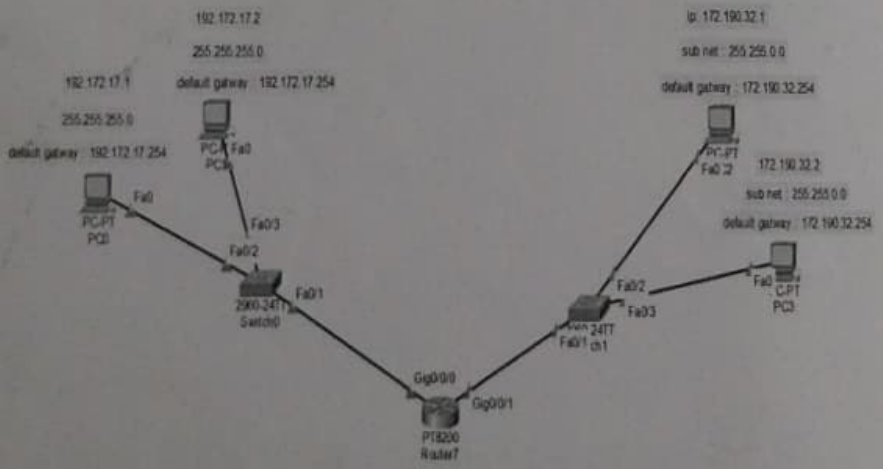
Code:

```
enable
configure terminal
interface FastEthernet 0/0
ip address <IP> <Subnet Mask>
no shutdown
exit
interface FastEthernet 0/1
ip address <IP> <Subnet Mask>
no shutdown
exit.
```

6. Configure default gateway of each PC with IP address of connected router Interface.

7. Use simulation Mode and send a PDU from one network to another.

8. Observe the packet path through the router and verify the successful communication.



```
Router>
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#
Router(config)#interface GigabitEthernet0/0/0
Router(config-if)# ip address 192.172.17.254 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# exit
Router(config)#
Router(config)#interface GigabitEthernet0/0/1
Router(config-if)# ip address 172.190.32.254 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# exit
Router(config)#
Router(config)#
Router(config)#
Router(config)#
Router(config)#
Router(config)#
Router(config)#
```

RESULT:

The Simple Subnetwork was successfully designed and configured on CISCO Packet Tracer and communication between different networks.